

FYR

A CDN-First Full-Stack Web Framework with Browser Python and Rust WebAssembly

Computer Science Project Design and Working Prototype

Version 0.1.0 | July 2026

Project name	Fyr (pronounced “fire”)
Project type	Open-source CDN-first web framework
Primary languages	JavaScript, Python and Rust
Target users	Students, independent developers, educators and small teams
Prototype status	Working reference implementation and production-oriented starter

Executive Summary

Fyr is a compact, CDN-first web framework that upgrades ordinary HTML into a reactive full-stack application without requiring a frontend build system. A developer adds a script tag, declares state and behavior with readable HTML directives, and connects to a secure Python backend. Optional plugins load Python into the browser through Pyodide and load precompiled Rust modules through WebAssembly. The design prioritizes speed, efficiency, progressive loading and low setup complexity.

The prototype demonstrates authentication, HTTP-only session cookies, SQLite persistence through SQLAlchemy, task CRUD operations, aggregate server actions, responsive production-style UI, browser-side Python execution and a real WebAssembly calculation. It is intended as a substantial foundation, not a claim that every operational control needed by a public production service is already complete.

1. Problem Statement

Modern web development often forces beginners and small teams to maintain several disconnected layers: an HTML interface, a JavaScript framework, a package manager, a bundler, a backend framework, API client code, duplicated validation models and deployment configuration. This can make a simple application difficult to understand and expensive to maintain.

- Build-tool complexity: many applications require Node.js configuration before a single interactive page can run.
- Frontend/backend duplication: endpoint names, data models and error handling are repeatedly implemented on both sides.
- Runtime inefficiency: large dependencies may load even when a page does not use them.
- Language isolation: Python learners cannot easily reuse their knowledge in browser interactions, while Rust performance normally requires specialist WebAssembly integration.
- Unsafe architecture choices: developers may place protected logic or secrets in browser code because the frontend/backend boundary is unclear.

2. Proposed Solution

Fyr uses progressive enhancement. The smallest page loads only the core JavaScript file. Python and Rust are separate plugins, so applications pay their download and initialization costs only when those runtimes are required. Protected logic remains on a backend, while CPU-heavy or educational calculations may run locally in the browser.

```
<script src="https://cdn.jsdelivr.net/npm/fyr-framework@0.1.0/dist/fyr.min.js"></script>
<div fyr-app="counter" fyr-controller="counter">
  <button fyr-click="count++">Increase</button>
  <strong fyr-text="count"></strong>
</div>
```

3. Project Objectives

- Enable advanced HTML applications with one CDN script and no mandatory compilation step.
- Provide reactive state, events, forms, loops, conditional rendering, HTTP calls and server actions.
- Give Python developers a secure backend path and an optional browser-Python path.
- Give Rust developers a standard WebAssembly plugin path for optimized browser computation.
- Minimize initial page weight through modular, lazy-loaded plugins.
- Maintain a clear security boundary between trusted server code and inspectable browser code.
- Offer a runnable example that can be studied, extended, tested and deployed.

4. System Architecture



5. Core Framework Design

5.1 Reactive State

The core wraps application state in JavaScript Proxy objects. A state mutation schedules one microtask-based render, preventing repeated immediate scans during a burst of updates. Nested objects and arrays are wrapped lazily and cached with WeakMap to avoid creating duplicate proxies.

5.2 HTML Directives

Directive	Purpose	Example
fyr-text	Render text from state	fyr-text="user.name"
fyr-show	Toggle visibility	fyr-show="authenticated"
fyr-model	Two-way form binding	fyr-model="draft.title"
fyr-click	Execute an event statement	fyr-click="save()"

fyr-submit	Submit without page refresh	fyr-submit="login()"
fyr-for	Repeat a template	fyr-for="task in tasks"
fyr-bind:*	Bind an HTML attribute	fyr-bind:disabled="busy"
fyr-class	Toggle CSS classes	fyr-class="{ high: urgent }"

5.3 HTTP and Server Actions

The HTTP client normalizes JSON requests, includes authentication cookies, parses errors and throws a structured `FyrHttpError`. Server actions provide named backend operations such as `dashboard.stats`, allowing aggregate or transactional behavior that does not fit simple CRUD.

```
const stats = await Fyr.action("dashboard.stats");
const task = await Fyr.http.post("/api/tasks", draft);
```

6. Bringing Python to the Browser

Fyr Python dynamically imports Pyodide from a CDN. Pyodide runs CPython compiled to WebAssembly. The plugin is lazy: the larger runtime is not downloaded until `Fyr.python.run()` is called. A small bridge exposes safe DOM helper functions. Browser Python is appropriate for education, numerical processing, demonstrations and offline calculations; it is not appropriate for secrets, protected authorization decisions or direct privileged database access.

```
const result = await Fyr.python.run("12 ** 2");
console.log(result); // 144
```

7. Bringing Rust to the Browser

Rust source is compiled ahead of time to WebAssembly and the resulting `.wasm` file is published with the CDN package. The browser fetches the binary, instantiates it with the WebAssembly API and stores its exports in a named module registry. The provided prototype includes a working binary exporting `add(i32, i32)`. The repository also contains a Rust `wasm-bindgen` project for extending the engine.

```
await Fyr.wasm.load("engine", "/fyr/fyr-engine.wasm");
const answer = Fyr.wasm.call("engine", "add", 20, 22); // 42
```

8. Backend Design

The reference backend uses FastAPI, Pydantic, SQLAlchemy and signed JWT session tokens stored in HTTP-only cookies. The backend validates all writes, queries trusted database values and never accepts browser claims as proof of authorization or payment. SQLite provides a simple local setup; PostgreSQL is the recommended production database.

Backend component	Responsibility
FastAPI routes	Authentication, health checks and CRUD endpoints

Pydantic schemas	Input validation and size constraints
SQLAlchemy	Database access and model mapping
JWT cookie	Tamper-resistant session identity
Action registry	Named business operations
Static mount	Serves the example and local CDN files

9. Working Prototype Features

- Responsive login and dashboard interface.
- Seeded administrator account for immediate testing.
- HTTP-only cookie authentication.
- Persistent task creation, listing, status update and deletion.
- Dashboard statistics through a named server action.
- Working WebAssembly addition module.
- Lazy browser Python square calculation.
- Health endpoint, unit test, Dockerfile, Render configuration and GitHub Actions workflow.
- npm-ready package metadata and jsDelivr/UNPKG entry fields.

10. Performance and Efficiency Strategy

- Modular loading: core, Python and WASM are separate files.
- Lazy Python initialization: Pyodide downloads only after explicit use.
- Native WebAssembly: optimized Rust routines avoid interpreting JavaScript for compute-heavy work.
- Microtask rendering: multiple state changes can collapse into a single render pass.
- Pinned CDN versions: applications gain reproducible assets and cache reuse.
- Backend pagination and database indexes should be added as datasets grow.
- Production builds should minify scripts, compress responses and use immutable cache headers.

11. Security Analysis

Every browser runtime is visible to the user. Therefore Fyr treats browser JavaScript, browser Python and Rust WebAssembly as untrusted presentation/computation layers. Authentication, authorization, billing, database credentials and private API keys stay on the backend.

- Use HTTPS and Secure cookies in production.
- Add CSRF tokens for cookie-authenticated state-changing requests.
- Replace the development secret and use managed secret storage.
- Add rate limiting at the application and reverse-proxy layers.
- Validate MIME type, file signature, size and storage path for uploads.
- Use a restrictive Content Security Policy; the current prototype expression evaluator uses Function and must be replaced by a safe parser before a hardened public release.
- Use Alembic migrations, audit logs, dependency scanning and external security review.

12. Testing and Evaluation

Test	Method	Expected result	Prototype status
WASM execution	Instantiate binary and call add(20,22)	42	Automated Node test passes
Backend syntax	Compile Python package	No syntax errors	Passes
Health API	GET /health	HTTP 200 and status ok	Included pytest
Authentication	Seed, login, read /api/auth/me	Session established	Implemented
CRUD	Create/update/delete task	Database reflects change	Implemented
Python browser	Run 12 ** 2	144	Requires internet access to Pyodide CDN

13. Limitations

- The framework is version 0.1.0 and does not yet provide a hardened expression parser or virtual DOM.
- Pyodide has a significant first-load cost and should remain optional.
- Rust must be compiled before CDN publication; browsers do not compile Rust source directly.
- The example uses SQLite and in-process action registration, which are not sufficient for every distributed system.
- No framework can make an application production-ready without operational controls, monitoring, backups and application-specific security review.

14. Future Development

- Safe expression parser and compiled directive cache.
- Web Components, slots and scoped component state.
- Client router with route guards and lazy page fragments.
- Automatic TypeScript clients generated from FastAPI OpenAPI schemas.
- PostgreSQL migrations and multi-tenant data isolation.
- Service-worker offline mode and local-first synchronization.
- Rust packages for image processing, compression and parser workloads.
- Python package manager controls and worker-thread execution.
- Official npm, PyPI, documentation and benchmark publishing.

15. Conclusion

Fyr demonstrates that a useful full-stack framework can remain approachable: one CDN script enhances HTML, a Python backend protects data and business logic, Python becomes an optional browser learning/computation tool, and Rust WebAssembly supplies efficient native-speed

modules. The project solves setup fragmentation and runtime overloading through a modular design, while preserving the critical distinction between public browser code and trusted server code.

Appendix A: Prototype Directory

```
fyr-framework/  
  dist/           CDN-ready scripts and WASM  
  packages/core/   Reactive JavaScript core  
  packages/python/ Pyodide bridge  
  packages/wasm/   WASM loader and registry  
  rust-engine/     Rust source  
  examples/production-dashboard/  
    app/           FastAPI backend  
    templates/     HTML application  
    static/        CSS and controller  
    tests/         Backend tests  
    docs/          Project document  
  .github/workflows/ Continuous integration
```

Appendix B: Startup Commands

```
cd examples\production-dashboard  
python -m venv .venv  
.\venv\Scripts\Activate.ps1  
pip install -r requirements.txt  
python -m app.seed  
uvicorn app.main:app --reload
```